

Functions in JS

```
function hello () {  
    console.log ("Hello " );  
}
```

Call function

```
hello ();
```

Functions with Arguments

value that we pass to the function

```
* function hello ( name ) {  
    console.log ( name );  
}
```

Parameter

```
hello ( " Aman " );
```

o/p : Aman

* For passing multiple arguments :

```
function hello ( name , age ) {  
    console.log ( ${name} ${age} );  
}
```

```
hello ( " Aman " , 23 ) ;
```

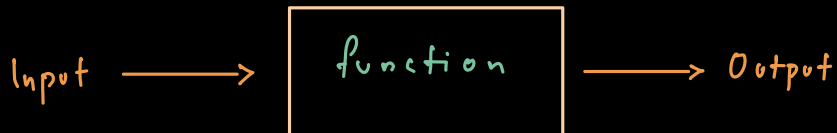
o/p : Aman , 23

```
hello ( " Aman " );
```

o/p : Aman , undefined

Return

* Used to return some value from the function.



```
function sum ( a, b ) {  
    return a + b ;  
}
```

```
sum ( 2, 7 ) ;
```

o/p : 9

* String can also be returned.

```
return "Garden" ;
```

Scope

* It determines the accessibility of variables, objects, and functions from diff. parts of the code

- o Function
- o Block
- o Lexical

1. Function Scope

variables defined inside a function are not accessible (visible) from outside the function.

```
let sum = 15 ;  
function Calsum ( a, b ) {  
    let sum = a+b ;  
}
```

// Global scope

↳ can be used
inside and outside
functⁿ

```
Console.log(sum) ;
```

o/p : sum beyond scope

2. Block Scope

variable declared inside a `{ }` block cannot be accessed from outside the block.

3. Lexical Scope

- A variable defined outside a function can be accessed inside another functⁿ defined after the variable declaration
- Opposite is not true.

```

function Outer () {
    let x = 5 , y = 6 ;

    function Inner () {           // function scope ✓
        console.log(x) ;
    }

    Inner () ;
}

```

But :

```

function Outer () {
    let x = 5 , y = 6 ;

    function Inner () {
        let a = 10 ;
        console.log(x) ;
    }

    Inner () ;
    console.log(a) ;           ✗ Not possible
}

```

Function Expressions

* Nameless functⁿ

* Value of functⁿ is stored in a variable

```

const sum = function (a, b) {
    return a + b ;
}

sum (2, 3) ;

```

o/p : 5

Higher Order Functions

A function that does one or both :

- 1) takes one or multiple functions as arguments
- 2) Returns a function

```
function multiGreet ( func, n ) {  
  for ( let i=1; i<=n; i++ ) {  
    func();  
  }  
}  
  
let greet = function () {  
  console.log("Hello");  
}  
  
multiGreet(greet, 2);
```

Methods

* functions defined under objects.

```
const calculator = {  
  add: function ( a, b ) {  
    return a + b;  
  },  
  sub: function ( a, b ) {  
    return a - b;  
  }  
}
```

Diagram annotations:

- object (points to the `calculator` variable)
- key (points to the `add` property name)
- value (points to the function assigned to `add`)
- // key: value pair (points to the `sub` property and its function)

SHORTHAND

```
const calculator = {  
  |  
    add( a, b ) {  
      |  
        return a + b ;  
      }  
    },  
    sub( a, b ) {  
      |  
        return a - b ;  
      }  
  }  
}
```

